McGill University
Department of
Mathematics and Statistics
Courtney Paquette

MATH 562 - Theory of Machine Learning

Project Description

Due Date: Tuesday, April 14 by 11:59 pm (local Montreal)
Submission Method: CrowdMark for the report and email with subject line `lastname firstname project code` and in the email, a list of all the group members names (first last and student numbers).

1 Project overview

The project for this course involves testing some of the theoretical results throughout this course with an emphasis on the overparameteried models in Chapter 12. The project is broken down into 2 components:

1. **Linear Regression and Joint Scaling Limits:** explore the concentration of the loss curves in joint scaling limits in (ridge) linear regression, generalization vs. training dynamics, and different choices for implementing SGD + other variants (streaming vs. multi-pass vs shuffling).

2. **Scaling Regimes in Two-Layer Neural Networks:** implement and analyze three fundamental training regimes for 2-layer neural networks (Neural Tangent Kernel, Mean Field, and Random Features).

While your group must tackle some aspect of each of these components, the project is meant to be open ended with extra projects for each of the components allowing you as a group to focus on one component that is of interest to the group. Consequently, the project is left open ended without very specific details. The key objective is to design and conduct experiments that either complement or disprove the practicality of the theoretical results. This is not a cookbook project with prescribed procedures. Instead, you will:

- Formulate hypotheses based on theory
- Design experiments to test these hypotheses
- Make informed choices about hyperparameters
- Interpret results and draw conclusions

1.1 Requirements:

In a group of no more than 5, you will be asked to

- Submit a 1 page project proposal due **March 11** via CrowdMark detailing who is on your team and plan for what each person will do. You will also need in 3-4 different (30 min) time slots that you can all attend for the in-person presentation during the week of **April 13-April 17**.
- Prepare a 20-25 min. (in-person) group presentation detailing your results from the project (Week of **April 13-17**). Be prepared to be asked questions regarding your set-up and the results.
- Submit 1 15 page report submitted via CrowdMark with code (submitted via email) detailing your results due **April 14**. A LaTeX template (`project_template.zip`) has been posted on myCourses for you to use to write your project report. You may go over the 10 pages (using the appendix), but the main results should appear in the 10 page report.

To receive a higher grade, you must go beyond just implementing the basic and doing the basic experiments. Doing just the basic experiments will give you only a B- on the project.

AI Policy. You may use AI tools to assist you on writing code for your final project. While I am allowing this, you will need to be able to explain what the AI did and the choices it made (During the presentation, you can not use AI). You are responsible for the output that the AI gives you. Moreover, I expect that you use the notation and material presented in this course.

2 Component 1: Linear/Random Feature Regression and Joint Scaling Limits

In this part of the project, we are interested in understanding the scaling limit (in a controlled setting) where both number of parameters d and number of sample n are large. Additionally, you will also explore the effect of batch sizes and learning rates as n, d change. For this problem, I will leave some of the details missing so that you will need to figure out mathematically what should be the correct initial scalings.

2.1 Mathematical Setup

Consider a least-squares problem:

$$\min_{\theta\in\mathbb{R}^d} \left\{\hat{\mathcal{R}}(\theta) = \tfrac{1}{2}\|\tfrac{1}{\sqrt{n}}A\theta - \tfrac{1}{\sqrt{n}}b\|_2^2\right\},$$

where $A \in \mathbb{R}^{n\times d}$ and $b \in \mathbb{R}^n$. Fix a $\theta^* \in \mathbb{R}^d$ such that $b = A\theta^* + \varepsilon$ where $\varepsilon \in \mathbb{R}^n$ is the label noise. Assume that

- Rows of A, $a_i \in \mathbb{R}^d$ $(i = 1, \ldots, n)$ are generated from a Gaussian with mean 0 and covariance Σ. (Start with $\Sigma = \alpha(d)I_{d\times d}$ (identity); you will need to figure out $\alpha(d)$)
- Entries of ε iid gaussians (independent of the way A is generated).
- Assume θ_0 (initialization for the SGD algorithm) is initialized randomly independent of A, ε.

You will explore the setting where $d, n \to \infty$ and $d = \rho \cdot n$ where $\rho \in (0, \infty)$. When $\rho > 1$, you are overparameterized and when $\rho < 1$, you are underparameterized.

What you need to do to start. Make the entries of θ_0, $\theta^* = O(1)$, e.g., if $\theta^*_j \sim N(0,1)$, then the entries of θ^* are order 1.

$$\textit{Goal: If both } d, n \to \infty \textit{ such that } d/n = \rho, \textit{ we want } \hat{\mathcal{R}}(\theta) = O(1).$$

In order to do this, you need to figure out

- If (entry of ε) $\varepsilon_i \sim N(0, \sigma^2\beta(n))$, what should $\beta(n)$ be?
- If (row of A) $a_i \sim N(0, \alpha(d)I_d)$, how should you choose $\alpha(d)$? For general covariance, if Σ is the covariance of a_i, then you can choose any $\Sigma = \alpha(d)\hat{\Sigma}$ (where $\alpha(d)$ is the same as in the identity covariance setting) such that $\|\hat{\Sigma}\|_{\text{op}} = O(1)$ and $\text{tr}\,(\hat{\Sigma}) = O(d)$.

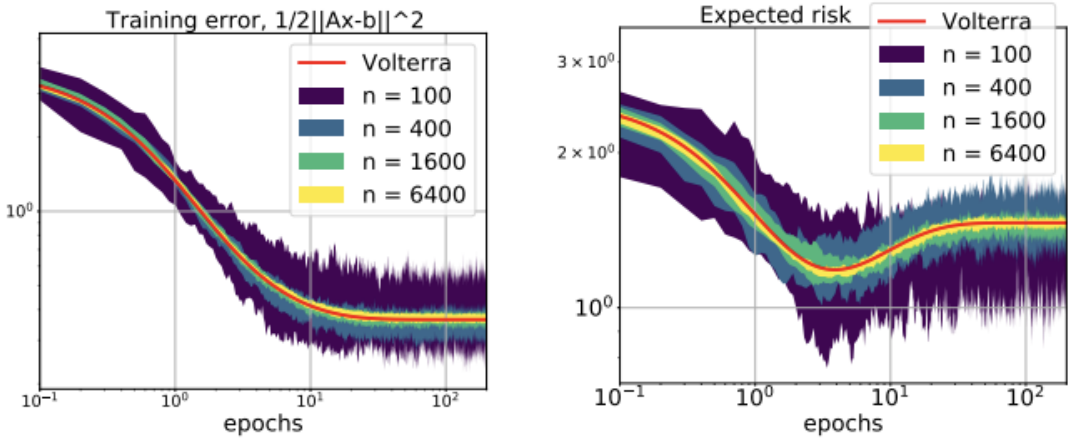
We are going to run *multi-pass* SGD on the the empirical risk, $\hat{\mathcal{R}}$, that is, at each iteration we generate we generate uniformly at random an index i_k from $1, \ldots, n$ and select the row $a_{i_k} \in \mathbb{R}^d$ and corresponding $b_{i_k} \in \mathbb{R}$, then update by

$$\theta_k = \theta_{k-1} - \gamma(d)\left[\tfrac{1}{n}(a_{i_k}^T\theta_{k-1} - b_{i_k})\right]a_{i_k}. \tag{1}$$

Here the learning rate $\gamma(d)$ may depend on d.

2.1.1 Experiments

Experiment 1: Figure out scaling on γ in small batch SGD. Fix a $\rho = d/n$ and vary $d = [100, 200, 400, 800, 1600]$. Create a hypothesis for what you think the learning rate $\gamma(d)$ should be and test your hypothesis. For the x-axis, instead of plotting with respect to iterations of SGD, plot epochs through the data set (i.e., iterations of SGD/n). If all your scalings are correct, you will see the concentration plots. Also plot the generalization error. What is the effect of σ^2 constant in the label noise? In the plots below, the SGD was run 10 times with different realizations of A and b. The variance (shaded region) across those 10 runs is plotted below.



Make sure that you optimize the absolute constant in the learning rate. Theory will often say that you should choose it to be roughly $1/\lambda_{\max}(A^TA)$. Is it true? Try different covariance structures Σ for the a_i. How do you choose your optimal absolute learning rate when the covariance changes? Create a hypothesis of what you think the absolute constant should be. (Hint: it is not the $1/\lambda_{\max}(A^TA)$ and design and experiment to verify your hypothesis.)

Experiment 2: Effect of randomness in SGD In this experiment, play around with the different ways of choosing the index i_k in SGD. For instance, consider the setting where you choose a random permutation of $1, \dots, n$ and iterate through. Then once you finish, you restart with the same permutation. Let's call this *single shuffle*. How about if you generate a new random permutation after doing n steps of SGD? Let's call this *multiple shuffle*. How do the dynamics of the algorithm change compared with choosing an index uniformly at random as in Experiment 1? Compare both training and generalization performance.

Experiment 3: Repeat Experiment 1 but with SGD with Momentum Recall the stochastic momentum algorithm discussed in class:

$$\theta_{k+1} = \theta - \gamma(n)\big[\tfrac{1}{n}(a_{i_k}^T\theta_k - b_{i_k})\big]a_{i_k} + \delta(\theta_k - \theta_{k-1}).$$

Here δ is an absolute constant independent of n.

Repeat Experiment 1.

Experiment 4: Small vs. Large Batch Sizes In this experiment, you will look at the effect of batch size. Here at each iteration generate a batchsize B of indices (without replacement) from $1, \dots, n$ uniformly at random and generate a stochastic gradient

$$g_k = \frac{1}{|B|}\sum_{i_k \in B}\frac{1}{n}\big[\frac{1}{n}(a_{i_k}^T\theta_{k-1} - b_{i_k}\big]a_{i_k}.$$

Then update (SGD) by

$$\theta_k = \theta_{k-1} - \gamma(d)g_k.$$

Suppose batch $|B| = \xi n^\eta$ where ξ and η are constant independent of n and d. Form a hypothesis about the value of η to make a distinction between *large batch* (i.e., looks like full-batch gradient descent) vs. *small batch* (i.e., looks like Experiment 1). You may also need to determine the order of $\gamma(d)$ in each setting. Design an experiment to test this.

Some references for this component.

- Chapter 4 and 5, `https://elliotpaquette.github.io/notes/high-d-limits-sgd-july2023.pdf`

Optional Experiment. Do the same for momentum but set $|B| = \xi n$ where $\xi \in (0, 1)$. A critical batchsize is such that adding more to the batch does not improve convergence. Do you see a "critical batch size" as you vary ξ?

For some references:

- See `https://openreview.net/pdf?id=z9poo2GhOh6`

3 Component 2: Scaling Regimes in Two-Layer Neural Networks

In this part of the project, you will implement and analyze three fundamental training regimes for two-layer neural networks: **Neural Tangent Kernel (NTK)**, **Mean Field (MF)**, and **Random Features (RF)**. The key objective is to **design and conduct experiments** that explore how different parameterizations lead to distinct training dynamics, learning rate requirements, and generalization behaviors.

3.1 Mathematical Setup

In this section, we describe a mathematical set-up for showing some of the results from theory (see also *Learning Theory from First Principles* by Francis Bach, Chapter 12).

3.1.1 Network Architecture

Consider a two-layer neural network with the following structure:

$$h(x;\theta) = \frac{1}{\alpha(m)} \sum_{i=1}^{m} a_i \sigma(w_i^\top x)$$

where:

- $x \in \mathbb{R}^d$ is the input
- $w_i \in \mathbb{R}^d$ are the first-layer (inner) weights
- $a_i \in \mathbb{R}$ are the second-layer (outer) weights
- $\sigma(\cdot)$ is the activation function
- m is the number of hidden neurons
- $\alpha(m)$ is a scaling factor that defines the parameterization regime

3.1.2 Three Scaling Regimes

The three regimes differ in their choice of $\alpha(m)$ and which parameters are trained:

1. **Mean Field (MF)**: $\alpha(m) = m$
    - Train both w_i and a_i
    - This is related to μP parameterization, see [Yang and Hu(2021), Bach and Chizat(2021)]
2. **Neural Tangent Kernel (NTK) or Lazy Regime**: $\alpha(m) = \sqrt{m}$
    - Train both w_i and a_i
    - Function h stays $O(1)$ but gradients are $O(1/\sqrt{m})$
    - References: [Chizat et al.(2019)Chizat, Oyallon, and Bach]
3. **Random Features (RF)**: $\alpha(m) = \sqrt{m}$
    - Train only a_i; keep w_i **fixed** at initialization
    - Equivalent to linear regression in a random feature space

3.1.3 Activation Functions

You must implement three activation functions:

ReLU (Rectified Linear Unit):

$$\sigma(z) = \max(0, z), \quad \sigma'(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{otherwise} \end{cases}$$

Error Function (erf):

$$\sigma(z) = \text{erf}(z) = \frac{2}{\sqrt{\pi}} \int_0^z e^{-t^2} dt, \quad \sigma'(z) = \frac{2}{\sqrt{\pi}} e^{-z^2}$$

Hyperbolic Tangent (tanh):

$$\sigma(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad \sigma'(z) = 1 - \tanh^2(z)$$

You may also play around with other activation functions if you want.

3.1.4 Student-Teacher Framework

Consider implementing a student-teacher framework. Here we generate our "ground truth" labels from a 2-layer neural network with possibly different hidden neurons. Then your objective is to minimizing a 2-layer network with $h(x; \theta)$ and mean squared error as your loss function, known as the student network.

Target Function (Teacher). Generate a "ground truth" function using a fixed neural network:

$$h^*(x) = \frac{1}{\alpha(m^*)} \sum_{i=1}^{m^*} a_i^* \sigma(w_i^{*\top} x).$$

Initialize w_i^*, a_i^* randomly and keep them fixed.

Training Data. To test the theory, let us generate samples/data using a known distribution. Generate n samples:

$$\{(x_j, y_j)\}_{j=1}^n \quad \text{where} \quad y_j = h^*(x_j)$$

For synthetic data, sample $x_j \sim \mathcal{N}(0, I_d/d)$ (Gaussian with covariance I_d/d). Here I_d is the $(d \times d)$-identity matrix.

Student Network. Let $v_i \in \mathbb{R}^{d+1}$ for $i = 1, \ldots, m$ and let $V = [v_1, \ldots, v_m]$ be the concatenation of all the weights. Initialize a network $h(x; V)$ with random weights and train it to minimize:

$$\min_V \left\{ \mathcal{R}(V) = \frac{1}{n} \sum_{j=1}^n (y_j - h(x_j; V))^2 \right\}.$$

You will solve this optimization problem using **gradient descent**,

$$V_{k+1} = V_k - \eta(m) \nabla \mathcal{R}(V).$$

Note, please make sure that you are scaled correctly. As you vary m, the empirical loss should not go grow or shrink.

Test Data. Generate a separate test set to measure generalization error. Make sure you do not train using this test set.

3.1.5 Implementation Requirements

Implement three classes corresponding to the three regimes. Each class should include:

- `__init__(d, m, activation, seed)`: Initialize weights randomly
- `sigma(z)` and `sigma_prime(z)`: Activation and derivative
- `forward(X)`: Compute predictions with appropriate scaling
- `backward(y_true, y_pred)`: Compute gradients via backpropagation
- `gradient_descent_step(X, y_true, learning_rate)`: One optimization step
- `train(X, y, learning_rate, n_iterations)`: Full training loop
- `mse_loss(y_true, y_pred)`: Compute mean squared error

3.1.6 Required Experiments

You must design and conduct three main experiments. For each, we provide the objectives and experimental guidelines, but **you** must make specific design choices. These are left up to you to make the decisions. You need to discuss these in your write-up and presentation.

Experiment 1: Performance Comparison Across Regimes. The objective is the following: Compare training and generalization performance of the three regimes (MF, NTK, RF) to understand:

- Which regime converges fastest?
- Which achieves lowest training/test error?
- How do different activations affect each regime?
- How does performance scale with network width m?

Some things to consider in your design choices include

- Choice of d, n, n_{test}(number of samples you use in your test set), m^* (for synthetic experiments
- Which values of m to test? (Suggested: 3-5 values that double)
- How many iterations to run gradient descent?
- Which learning rates for each regime (Use your intuition from Chapter 12)?
- How to evaluate the test loss/generalization performance?

Suggested Starting Point

Conservative starting configuration:

- $d = 10$, $n = 3000$, $n_{\text{test}} = 500$, $m^* = 20$
- $m \in \{100, 200, 400, 800\}$
- Test all three activations (ReLU, erf, tanh)
- 1000 iterations

Then explore:

- Does increasing m improve generalization?
- Which activation works best for each regime?
- Can you get better results by tuning learning rates?

Extensions to Explore. If you want to go beyond the synthetic datasets, you can also look at real datasets and perform experiments on these. Here are two examples and how to download them.
MNIST: 28×28 grayscale images, 10 classes

```
from sklearn.datasets import fetch_openml
X, y = fetch_openml('mnist_784', version=1,
                    return_X_y=True, as_frame=False)
```

CIFAR-10: 32×32 RGB images, 10 classes

```
from torchvision.datasets import CIFAR10
dataset = CIFAR10(root='./data', train=True,
                  download=True, transform=transforms.ToTensor())
```

Hints for Real Data

Simplifications you can make:

- Binary classification: Pick two classes (e.g., 0 vs. 1)
- Subset: Use 5000-10000 samples instead of full 60k
- Preprocessing: Flatten and normalize data (mean 0 with standard deviation 1)

Design questions to consider:

- Do the relative performances of regimes differ from synthetic data?
- Do you need larger m for real data?
- Which activation works best on real data?

Examples of what can be reported. These are some ideas of what to report or show. You do not need to show all of these, but these give you some ideas on what do to.

- Table of final train/test losses for all configurations
- Training curves (loss vs. iteration) showing convergence
- Test error vs. m to show scaling behavior
- Comparison across activations
- Discussion of which regime performs best and why

Experiment 2: Learning Rate Scaling Analysis The main objective in this experiment is to test the central hypothesis of how the learning rates should scale according to theory, that is, *The correct learning rate scaling makes training dynamics independent of network width* m. If we are running gradient descent,

$$V_{k+1} = V_k - \eta(m)\nabla\mathcal{R}(V_k),$$

How should we choose η as a function of m? Specifically, we saw in class that

- Mean Field: $\eta \propto m$
- NTK: $\eta =$ constant, independent of m
- Random Features: $\eta =$ constant, independent of m

Note there are constants involved which are not specified and depend on the activation function. Moreover the results presented in class only hold when $m \to \infty$ so you might see some finite dimensional effects when m is not ∞.

For each regime (Mean Field, NTK, Random Feature), you will need to:

1. Determine a good base learning rate β; this is the constant and should be independent of m.
2. Test 3-4 different scalings: $\eta = \beta \cdot m^{\alpha}$ for different α; You should test α's what are both too large and too small to illustrate different effects.
3. Train with multiple values of m.

Before you start on this, you need to think about what you expect to happen, that is, what should the plots look like when you get the correct scaling? and what should (theoretically) happen when you choose a scaling which is too small or too large? Does your intuition manifest itself in the plots? If not, can you explain why?.

Suggested Experimental Protocol

For Mean Field:

1. Fix $d = 10$, $n = 3000$, $m^* = 200$, activation $=$ tanh
2. Choose $m \in \{100, 200, 400, 800, 1600, 3200\}$
3. Test scalings:
   - $\eta = \beta \cdot m$ (predicted correct)
   - $\eta = \beta \cdot \sqrt{m}$
   - $\eta = \beta$ (constant)
   - $\eta = \beta \cdot m^2/100$
4. Don't need full convergence to see the scaling

For NTK:

1. Same d, n, m^*, m values
2. Test scalings:
   - $\eta = \beta$ (predicted correct)
   - $\eta = \beta \cdot \sqrt{m}/2$
   - $\eta = \beta \cdot m/10$
   - $\eta = \beta/\sqrt{m}$

For Random Features:

1. Same approach as NTK

You can (and should) test other scalings.

Note for reproducibility and fair comparison you will need to pay attention to random seeds.

Critical Detail: Reproducibility

For fair comparison across m values:

- Use the **same random seed** for initializing networks with different α
- This ensures differences in curves are due to scaling, not random initialization
- Example:

```
for m in m_values:
    nn = TwoLayerNN(d, m, activation='tanh', seed=100)
    # All networks start from similar random initialization
```

Quantitative Analysis: Measuring Overlap It's not enough to just plot curves—you need a **quantitative metric**! One such metric would be to compute the variance across different m values: At each iteration t, compute variance:

$$\text{Var}_t = \frac{1}{|M|} \sum_{m \in M} (\mathcal{R}_t^{(m)} - \bar{\mathcal{R}}_t)^2$$

where $\bar{\mathcal{R}}_t = \frac{1}{|M|} \sum_{m \in M} \mathcal{R}_t^{(m)}$ is the mean loss at iteration t. Then the **overall metric** would be

$$\text{Mean Variance} = \frac{1}{T} \sum_{t=1}^{T} \text{Var}_t$$

Interpretation: We can interpret the metric by the following

- **Low variance** (e.g., $< 10^{-4}$): Curves overlap well $\rightarrow$ correct scaling
- **High variance** (e.g., $> 10^{-2}$): Curves diverge $\rightarrow$ wrong scaling

Be careful that this metric is not fully representative of whether you have the correct scaling! You are also welcome to come up with other metrics. An example of the implementation of this metric is illustrated below:

```
loss_array = np.array([losses[m] for m in m_values])
# Shape: (n_m, n_iterations)
variance_over_time = np.var(loss_array, axis=0)
mean_variance = np.mean(variance_over_time)
```

Examples of things one may report. For each of the different scaling regimes:

- Figures showing training curves for each α
- Table of mean variance for each scaling
- Discussion: Does the empirical result match the theoretical prediction?

Experiment 3: Kernel Evolution and Constancy The key objective in this experiment is: *Demonstrate that NTK and RF kernels remain approximately constant during training, with RF being exactly constant.* From class, we learned theoretically that

- **Random Features**: Kernel is **exactly constant** (since w is fixed)
- **NTK**: Kernel is **approximately constant** (changes decrease as $m \rightarrow \infty$)
- **Mean Field** (optional): Kernel changes significantly

Kernel Definitions. To remind everyone, recall the definitions of neural tangent kernel and random features from class.

Neural Tangent Kernel (NTK): For $x, x' \in \mathbb{R}^d$ (two samples),

$$K^{\text{NTK}}(x, x') = \left\langle \nabla_V h(x; V), \nabla_V h(x'; V) \right\rangle$$

For two-layer network:

$$K^{\text{NTK}}(x, x') = K_a(x, x') + K_w(x, x')$$

where:

$$K_a(x, x') = \frac{1}{m} \sum_{i=1}^{m} \sigma(w_i^\top x)\sigma(w_i^\top x') \quad \text{and} \quad K_w(x, x') = \frac{1}{m} \sum_{i=1}^{m} a_i^2 \sigma'(w_i^\top x)\sigma'(w_i^\top x')(x^\top x'). \tag{2}$$

Random Features Kernel:

$$K^{\text{RF}}(x, x') = \frac{1}{m} \sum_{i=1}^{m} \sigma(w_i^\top x)\sigma(w_i^\top x').$$

Implementation Guidelines. I'm providing some help with computing the kernels. You do not need to use these.

Efficient Kernel Computation

For NTK:

```
def compute_ntk_matrix(nn, X):
    n, m = X.shape[0], nn.m

    # Forward pass
    pre_activation = X @ nn.w.T  # (n, m)
    hidden = nn.sigma(pre_activation)  # (n, m)
    hidden_prime = nn.sigma_prime(pre_activation)  # (n, m)

    # K_a contribution
    K_a = (1.0 / m) * (hidden @ hidden.T)

    # K_w contribution
    X_inner = X @ X.T  # (n, n)
    a_sq = nn.a ** 2
    weighted_hprime = hidden_prime * a_sq[None, :]
    K_w = (1.0 / m) * (weighted_hprime @ hidden_prime.T) * X_inner

    return K_a + K_w
```

For Random Features:

```
def compute_rf_kernel(model, X):
    pre_activation = X @ model.w.T
    features = model.sigma(pre_activation)
    K = (1.0 / m) * (features @ features.T)
    return K
```

Some additional help in this experiment as this can get costly very fast. Fix a network configuration to start (choose an m, d, and activation function). For computing the kernel, use the data points that you are using in training $X \in \mathbb{R}^{n \times d}$ and make sure that n is relatively small since the

kernel is an $n \times n$ matrix. You should not compute the kernel at every iteration (this is too costly!) - instead compute the kernel every once and while through training.

There are many thing you many want to show. For instance, you can do a 'heat map' showing the values of $K(x, x')$ for two samples, x, x'. Some additional metrics you may want to track include:

- Frobenius norm: $\|K\|_F = \sqrt{\sum_{i,j} K_{ij}^2}$
- Relative change: $\frac{\|K_t - K_0\|_F}{\|K_0\|_F}$
- Absolute change: $\|K_t - K_0\|_F$
- Mean kernel value: $\frac{1}{n^2} \sum_{i,j} K_{ij}$

An Example of the Experimental Step-Up for Kernel

Setup:

- $d = 10$, $m = 400$, $n_{\text{kernel}} = 50$, activation $=$ tanh
- Learning rate: Use appropriate scaling from Experiment 2
- Checkpoints: $[0, 25, 50, 75, 100]$ for computing the kernel

Extensions to Explore (Optional).

Optional Deeper Investigations

Interesting questions you could explore:

- Does NTK kernel become more constant as m increases? Test $m \in \{100, 200, 400, 800\}$
- How does kernel change depend on learning rate?
- Compare with Mean Field: Does the kernel change significantly?
- Is there a relationship between kernel change and generalization error?
- How do kernel eigenvalues evolve during training?

4 Additional Information about the Project

4.1 Project report

You are required to submit a written report with your software containing the following:

- Report should not exceed 15 pages (you may put some figures in an Appendix which does not count against your page limit total; however the 15 pages should be a complete report (i.e. it should contain figures and reflect what you want the reader to learn from your experiments). On **page 16**, include a list of each group member (full name and student idea) with a paragraph of exactly what each person did.
- You do not need to include an introduction or abstract.
- Summarize each component. Provide some background information about each component and explicit set-up.
- Provide a detailed explanation of the experimentations, metrics, and numerical set-up that you used for each component.
- Summarize your finding and emphasize interesting things you saw.
- Provide tables of the different input paramters you used in your code (e.g, number of iterations, default learning rates, number of hidden neurons, etc).
- Provide visual interpretations (*e.g.* graph of the function values relative to the iterations, etc) of the results.
- Comment on the results. Provide detailed description of your findings. How did the experiments on synthetic data (or real data) match the theoretical findings? If some did not, comment on why you think there were disagreements and where they might have come from.
- Summarize. Comment on any difficulties or limitations you faced.
- I emphasize again this project is meant to be open-ended and you should feel free to explore beyond the confines of what is listed above.

4.2 Project Presentation

The 20-25 minute presentation should just be a summary of your write-up. You should mainly focus on in the presentation on your results and your numerical simulation set-up.

Project grading

I am looking for you to go beyond just implementing and doing the basic analysis of each component. I want you to explore at least one component. For completing the project without doing any extra work will only get you a B- on the project. This project is meant to be open ended allowing you to design experiments and metrics to test the theoretical results. If your results do not match what the theory suggests, think about why and try to understand what is making them different. Do NOT be afraid to try things even if they don't work. You can add these failures to your discussion in the report/presentation.

Please ask me if you have any questions about my expectations or anything else!

(Finally, note that you are expected to work on the code and report for this project *in your own group*. If there is any evidence of sharing code or writing in your report, beyond your group, then there will be *serious* unfortunate consequences.)

References

[Bach and Chizat(2021)] Francis Bach and Lénaïc Chizat. Gradient descent on infinitely wide neural networks: Global convergence and generalization. *arXiv preprint arXiv:2110.08084*, 2021.

[Chizat et al.(2019)Chizat, Oyallon, and Bach] L. Chizat, E. Oyallon, and F. Bach. On lazy training in differentiable programming. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[Yang and Hu(2021)] Greg Yang and Edward J Hu. Tensor programs iv: Feature learning in infinite-width neural networks. In *International Conference on Machine Learning*, pages 11727–11737. PMLR, 2021.